Évaluation OpenGL 2026

Sujet 1 : Poissons dans un aquarium

Pour ce sujet, il n'y a pas besoin d'aller plus loin dans le TP que là où nous nous sommes arrêtés (correction v6).

On fournit un code qui permet de simuler les déplacements des poissons dans un aquarium. Vous devez reprendre ce code, et afficher l'aquarium, les poissons, et faire évoluer les poissons dans l'aquarium en temps réel. On demande également de créer un mode de contrôle où la camera suit les mouvements d'un poisson choisi au hasard, et s'oriente selon la direction du poisson. On pourra changer de contrôle en appuyant sur une touche donnée.

Explication du code fourni

Le code fourni est reparti sur deux classes.

- La classe `Poisson` décrit le comportement d'un poisson, et notamment la façon dont le poisson change sa direction de déplacement selon la position des autres poissons..
    - Les arguments de la classe `Poisson` :

nom de l'attribut et visibilité	**type de l'attribut**	**description**
`position` (protected)	`glm::vec3`	La position du poisson.
`vitesse` (protected)	`glm::vec3`	La vitesse du poisson. L'orientation de la vitesse peut changer, mais sa norme vaut toujours 1.
`close_range` (protected)	`float`	(Attribut statique) Définit le rayon d'un cercle autour du poisson. Si d'autres poisson se trouvent dans ce cercle, les poissons vont avoir tendance à s'éloigner.
`far_range` (protected)	`float`	(Attribut statique) Définit le rayon d'un second cercle autour du poisson, plus grand que celui définit par `close_range`. Si des poissons sont dans ce second cercle, mais en dehors du premier, alors ils vont avoir tendance à se rapprocher.

    - Les méthodes principales de la classe `Poisson` :

nom de la méthodes	arguments	type de retour	description
`Poisson`	**position (`glm::vec3`)** : La position initiale du poisson | **vitesse (`glm::vec3`)** : La vitesse initiale du poisson	`void`	Le constructeur.
`eviter_mur`	**dimensions_aquarium (`glm::vec3`)** : Les dimensions de l'aquarium dans lequel est le poisson.	`bool`	Si le poisson s'apprête à rentrer en collision avec un des murs de l'aquarium, lui fait faire demi-tour. On renvoi `true` si un mur a été évité.
`eviter_poisson`	**poissons (`vector<Poisson>`)** : La liste des poissons de l'aquarium | **dimensions_aquarium (`glm::vec3`)** : Les dimensions de l'aquarium dans lequel est le poisson.	`bool`	Si d'autres poissons se trouvent dans le cercle `close_range`, le poisson change sa trajectoire pour éviter le plus proche. On renvoi `true` si un poisson a été évité.
`aligner`	**poissons (`vector<Poisson>`)** : La liste des poissons de l'aquarium	`void`	Si des poissons se trouvent à une distance entre `close_range` et `far_range`, change la trajectoire du poisson pour aligner sa vitesse sur eux.
`deplacement`	**poissons (`vector<Poisson>`)** : La liste des poissons de l'aquarium | **dimensions_aquarium (`glm::vec3`)** : Les dimensions de l'aquarium dans lequel est le poisson.	`void`	Fait le déplacement du poisson, avec les priorités suivantes : on évite les murs en priorité s'il y en a, sinon on évite les poissons à éviter s'il y en a, et sinon on s'aligne avec les autres poissons.
`get_position`	/	`glm::vec3`	Renvoi la position du poisson.

nom de la méthodes	arguments	type de retour	description
`get_vitesse`	/	`glm::vec3`	Renvoi la vitesse du poisson. Cette vitesse est toujours normalisée.

- La classe `Aquarium` décrit un aquarium contenant un certain nombre de poissons.
  - Les arguments de la classe `Aquarium` :

nom de l'attribut et visibilité	type de l'attribut	description
`dimensions_aquarium` (privé)	`glm::vec3`	Les dimensions de l'aquarium. L'aquarium est un pavé, on donne donc ici sa longueur (axe x), sa hauteur (axe y) et sa profondeur (axe z).
`liste_poissons` (privé)	`vector<Poisson>`	La liste des poissons se trouvant dans l'aquarium.

  - Les méthodes principales de la classe `Aquarium`:

nom de la méthodes	arguments	type de retour	description
`Aquarium`	**n (`int`)** : Le nombre de poissons dans l'aquarium | **dimensions_aquarium (`glm::vec3`)** : Les dimensions de l'aquarium	`void`	Le constructeur. Il crée une liste de `n` poissons en attribuant des positions et vitesses aléatoires.
`update`	/	`void`	Fait bouger tout les poissons du banc.
`get_dimensions`	/	`glm::vec3`	Retourne les dimensions de l'aquarium. L'aquarium est un pavé, on donne donc ici sa longueur (axe x), sa hauteur (axe y) et sa profondeur (axe z).
`get_poissons`	/	`vector<Poisson>`	Retourne la liste des poissons se trouvant dans l'aquarium.

Pour créer un aquarium puis faire bouger les poissons, le code à utiliser ressemble à l'exemple ci-dessous :

```
Aquarium aquarium(100, glm::vec3(100,60,50)); //On crée un aquarium contenant 100 poissons,
et avec une longueur de 100, une hauteur de 60 et une profondeur de 50.
for (int i=0; i<100; i++){
    aquarium.update();
}
```

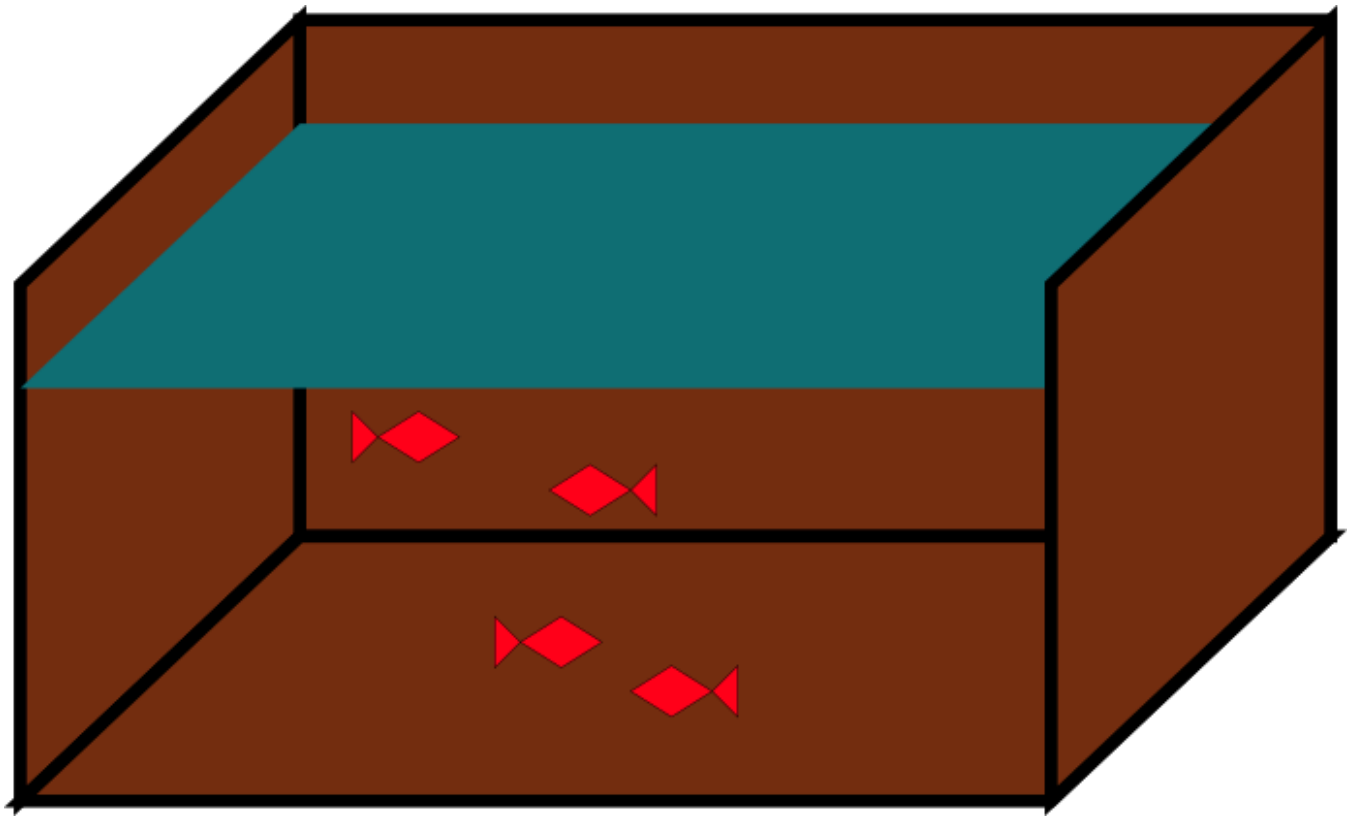
Objectifs :

- Créer un objet de type `Aquarium` contenant au minimum 20 poissons, et afficher dans la scène tout les murs de l'aquarium, sauf l'un des murs latéraux et celui du haut (voir le schéma du rendu attendu).
- Créer un plan avec une texture d'eau, qui représentera la surface de l'eau.
- Créer un objet représentant un poisson , puis afficher cet objet une fois pour chaque poisson. Pour représenter le poisson, on fournit une texture, des coordonnées de vertex buffer et d'uv buffer dans le dossier `poisson`. Vous êtes libres de les utiliser, ou d'en utiliser d'autres.
- Faire évoluer les poissons en temps réel.
- Créer un mode de contrôle où la camera suit un poisson choisi au hasard, et où l'utilisateur ne peut pas bouger ou tourner la camera. La camera devra être orientée comme le poisson, et tourner avec lui.
- Donner la possibilité à l'utilisateur de passer d'un contrôle classique au contrôle "suivi de poisson" en appuyant sur une touche (par exemple la touche `ctrl`).

Bonus :

- Ajouter une classe `prédateurs` au code fourni. Cette classe héritera de la `poisson`, mais aura un comportement différent :
    - Le prédateur sera légèrement plus lent que les poissons classiques.
    - Si un poisson classique est proche, le prédateur s'approche de lui. Sinon, le prédateur se comporte comme un poisson classique.
    - Si un prédateur est à une distance inférieure d'un poisson que la distance qu'il parcourt en un tour, il mange ce poisson, qui disparait.
- Affichez les prédateurs, avec une autre texture que les poissons classiques.

Schéma du rendu attendu :



Sujet 2 : Simulateur d'avion

Pour ce sujet, il faut commencer par terminer le TP au moins jusqu'à la partie sur les lumières.

Il faudra ensuite afficher un avion, un plan qui représentera la terre, et une source lumineuse qui tournera autour de la sphère, qui représentera le soleil.

Objectifs :

- Créer un grand plan carré (au moins 100 unités de longueur), et lui appliquer une texture représentant une carte de monde.
- Créer un objet avec la forme d'un avion, avec une texture de votre choix, et placer cet objet au centre du plan. La forme de l'avion devra être importée depuis un ficiher `obj`.
- Créer une lumière ponctuelle à bonne distance du plan (au moins à 1000 unités du plan), et faire tourner cette lumière autour du plan. Il faudra faire attention à ce que la puissance de la lumière soit suffisamment élevée pour éclairer le plan.
- Créer un mode de contrôle où la camera se trouve un peu derrière et au dessus de l'avion, et suit ses mouvements. Dans ce mode de contrôle, l'utilisateur utilise les touches pour modifier la vitesse et l'orientation de l'avion. L'avion devra avoir une vitesse minimum en dessous de laquelle il ne peut pas descendre, et également un angle vertical maximal et minimal en dessous duquel il ne peut pas aller. Les touches utilisées seront :

Touche	Action
`z` ou ↑	Augmenter la vitesse

Touche	Action
`s` ou ↓	Diminuer la vitesse (ne pas descendre en dessous de la vitesse minimale)
`q` ou ←	Rotation vers la gauche
`d` ou →	Rotation vers la droite
`espace`	Rotation vers le haut (ne pas tourner au delà de la rotation maximale)
`shift`	Rotation vers le bas (ne pas tourner en dessous de la rotation minimale)

Bonus :

- Ajouter une interface utilisateur (voir la partie du TP sur les interfaces utilisateur) permettant à l'utilisateur de :
    - Modifier la couleur de l'avion (ce qui modifiera la couleur de la texture).
    - Modifier la puissance de la lumière ponctuelle
    - modifier le moment de la journée (c'est à dire la position de la lumière sur son orbite)

Rendu le 04/12